

Conception et Implémentation d'une Intelligence Artificielle pour le Jeu d'Escampe

Mathieu WAHARTE

Raphaël BINDA

Mai 2026

Résumé

Ce rapport détaille la conception et la mise en oeuvre d'une IA capable de jouer au jeu d'Escampe avec contraintes de temps total. Nous y présentons nos choix d'implantation, notamment les structures de données optimisées (Bitboards), les algorithmes de recherche (Negamax avec élagage Alpha-Beta, Iterative Deepening) ainsi que la fonction d'évaluation heuristique développée pour modéliser la complexité topologique du plateau. Nous abordons également les stratégies de placement initial, la gestion dynamique du temps d'exécution, et fournissons une analyse des performances validées par des tests rigoureux.

Table des matières

1	Introduction	3
1.1	Fin de Partie	3
1.2	Analyse de la Complexité	3
2	Choix d'Implantation et Structures de Données	3
2.1	Structure Générale du Projet	4
2.2	Représentation du Plateau (Bitboards hybride)	4
2.3	Optimisations d'Implémentation	5
3	Algorithmes de Recherche du Meilleur Coup	5
3.1	Negamax et Élagage Alpha-Beta	5
3.2	Approfondissement Itératif (Iterative Deepening)	5
3.3	Ordonnancement des Coups et Élagage Sélectif	6
4	Fonction d'Évaluation (Heuristiques)	6
4.1	Heuristiques Implémentées et Testées	7
5	Exploration d'une Évaluation par Réseau de Neurones : BandDPER	7
6	Stratégie Initiale : Placement et Ouvertures	16
6.1	Principes de Placement (Opening Book)	16
6.2	Stratégie par Phase de Jeu	16

7	Gestion du Temps Réel	16
8	Analyse des Performances et Tests	17
8.1	Évaluation Elo et Test SPRT	17
8.2	Validation de la Conduite Autonome	18
9	Bilan des Fonctionnalités et Statut du TODO	18
9.1	Fonctionnalités Entièrement Implémentées et Validées	18
9.2	Fonctionnalités Conçues, Rejetées ou Non Retenues	19
9.3	Perspectives et Travaux Futurs (TODO Ouverts)	19
10	Conclusion et Difficultés Rencontrées	20

1 Introduction

Dans le cadre de ce projet, l'objectif principal est de développer une Intelligence Artificielle capable de conduire de façon autonome une partie d'Escampe en temps limité (300s à 900s par joueur). L'agent doit communiquer avec un arbitre validant les coups et garantissant le respect des contraintes temporelles. Le jeu d'Escampe se caractérise par des règles de déplacement atypiques (dépendance à la bande ciblée par l'adversaire) et une asymétrie de la topologie du plateau. Ces spécificités requièrent des choix algorithmiques pointus, s'écartant parfois des implémentations classiques des jeux à information parfaite.

1.1 Fin de Partie

Une partie se termine dès qu'un joueur capture la licorne adverse en y déplaçant un paladin. Aucun score partiel n'existe, seule la capture compte. Pour les agents, il convient d'ajouter un mécanisme de détection de répétition de position afin d'éviter les boucles infinies.

1.2 Analyse de la Complexité

Facteur de branchement. Chaque pièce peut se déplacer dans au plus 4 directions lors de son premier saut, puis 3 directions pour les suivants (sans revenir sur ses pas), avec jusqu'à 3 sauts selon sa bande. Dans le pire des cas, avec 6 pièces :

$$b_{\max} = 6 \times (4 + 2 \times 3) = 60 \text{ coups légaux par tour.}$$

En pratique, la contrainte de bande, les bords du plateau et les blocages ramènent ce facteur à 15-25 coups légaux en position typique de milieu de partie, ce qui justifie l'emploi d'un élagage Alpha-Beta agressif.

Majorant du nombre d'états. En modélisant un état de jeu par les positions des 12 pièces parmi 36 cases et la bande imposée par le dernier coup (4 valeurs), on obtient :

$$\mathcal{M} = \binom{36}{1} \binom{35}{5} \binom{30}{1} \binom{29}{5} \times 4 = \frac{36!}{(5!)^2 (36-12)!} \times 4 \approx 1,67 \times 10^{14}$$

états distincts. Ce chiffre confirme que toute exploration exhaustive est impossible et que les heuristiques et l'élagage sont indispensables.

Sources de difficulté. Au-delà de la complexité combinatoire, les principales difficultés algorithmiques sont : (i) la contrainte de bande qui conditionne les pièces jouables au tour suivant, créant des effets de tempo et de forçage, (ii) l'absence de simplification rapide (pas de capture de pions), qui allonge les séquences tactiques et (iii) l'importance du placement initial qui engendre une forte variabilité de départ.

2 Choix d'Implantation et Structures de Données

Afin de garantir des performances optimales lors de l'exploration de l'arbre de jeu, les structures de données sous-jacentes doivent permettre une génération de coups et une évaluation d'états extrêmement rapides.

2.1 Structure Générale du Projet

Le code source est structuré en plusieurs packages modulaires favorisant la séparation des responsabilités et le découplage des composants :

- **Package interfaces** : Définit les contrats essentiels pour l'ensemble du projet : `IJoueur` (contrat requis par l'arbitre), `IBoard` (abstraction du plateau), `IMove` (représentation générique d'un coup), `IRole` (rôle du joueur), et `IHeuristic` (contrat pour les fonctions d'évaluation heuristique).
- **Package game** : Contient les implémentations physiques et conceptuelles du jeu. `EscampeBoard` modélise l'état et les transitions du plateau par le biais de Bitboards, `EscampeMove` représente un déplacement (coordonnées de départ, intermédiaires, d'arrivée et contrainte de bande), et `EscampeAIPlayer` implémente le joueur autonome, gérant l'approfondissement itératif et la délégation de la recherche.
- **Package algorithms** : Regroupe la logique de décision. Il se subdivise en :
 - `algorithms.search` : Implémente les algorithmes de recherche (`NegamaxKH`, `AlphaBetaKH`) ainsi que la classe d'ordonnancement `MoveOrderer` (Killer Moves + History Heuristic).
 - `algorithms.evaluation` : Contient la fonction d'évaluation linéaire (`Heuristic`, `HeuristicConfig`).
 - `Opening` : Gère la bibliothèque de départ (ouvertures pré-calculées).
 - `TimeManager` : Calcule dynamiquement le budget temporel restant.
- **Package model** : Contient la classe `DataExport` chargée de sérialiser et d'exporter les états de jeu joués au format JSON. Ces données d'apprentissage alimentent le pipeline d'entraînement Python du modèle de réseau de neurones BandDPER.
- **Package ranking** : Dédié à la validation et aux tournois comparatifs. Il contient `TournamentRunner` (simulation locale en parallèle), `AIRankingSystem` (suivi des classements Elo et tests statistiques SPRT) et `SimplexTuner` (implémentation de l'optimisation Simplex pour l'heuristique).
- **Package io** : Gère les interfaces d'exécution. `ClientJeu` assure la communication réseau TCP avec le serveur de l'arbitre, `Solo` permet de lancer une partie locale directe entre deux bots pour le débogage et `Applet` fournit le rendu graphique visuel.

2.2 Représentation du Plateau (Bitboards hybride)

Nous avons opté pour une représentation du plateau basée sur un tableau de `char` de 36 cases ainsi que des *Bitboards* [2]. Les bitboards sont utilisées pour calculer rapidement les coups légaux, les collisions et les menaces, tandis que le tableau de caractères sert à au stockage de l'état de manière simplifiée.

- Les opérations de manipulation de pièces (génération des coups, détection de collisions) se réduisent à des opérations bit à bit (ET, OU, décalages), exécutées en un seul cycle d'horloge.
- Cette approche permet d'éliminer les itérations coûteuses sur des tableaux multidimensionnels et minimise la création d'objets, limitant ainsi la pression sur le *Garbage Collector*.

2.3 Optimisations d'Implémentation

Plusieurs choix techniques ont été faits pour maximiser le nombre de noeuds explorés par seconde :

- **Stratégie Make-Unmake** (annulation de coup) : plutôt que de cloner le plateau à chaque noeud de l'arbre, les coups sont joués sur le plateau courant puis défaits après évaluation (`board.undo(move, role)`). Cette approche élimine toute allocation mémoire dans la boucle de recherche et réduit la pression sur le *Garbage Collector*.
- **Suppression des appels `instanceof`** lors de la génération des coups via des méthodes dédiées par type de pièce, évitant le coût du dispatch polymorphique à chaque noeud.
- **Pré-allocation des structures** : les tableaux de positions de pièces (paladins, licornes) et les masques de Bitboards sont alloués une seule fois. Le tri de la liste des coups est réalisé en place (*insertion sort* sans copie, cf. section 3.3).
- **Table de transposition (Zobrist Hashing)** [10] : le schéma d'encodage (XOR de valeurs aléatoires 64 bits par pièce et par case, entrées de table stockant profondeur, type de noeud, score et meilleur coup) a été entièrement conçu mais non intégré dans la version finale par contrainte de temps. Elle constitue l'optimisation la plus prometteuse pour des versions futures, pouvant notamment permettre la détection de répétition de position.

3 Algorithmes de Recherche du Meilleur Coup

La recherche du meilleur coup repose sur la théorie de la recherche adversarielle pour les jeux à somme nulle.

3.1 Negamax et Élagage Alpha-Beta

Nous avons implémenté l'algorithme Negamax (une variante élégante du Minimax) couplé à un élagage Alpha-Beta [1]. Cette approche unifie l'évaluation pour les deux joueurs, réduisant ainsi la complexité du code. L'élagage Alpha-Beta permet de réduire drastiquement le facteur de branchement effectif de l'arbre de recherche en ignorant les branches dont il est prouvé qu'elles sont sous-optimales. Avant cela, nous avons expérimenté avec Minimax classique puis Alpha-Beta simple mais la version Negamax fut la plus performante. Un problème rencontré avec Negamax fut les `Integer.MIN_VALUE` et `Integer.MAX_VALUE` utilisés pour α et β : lors de l'inversion de signe, cela causait des dépassements d'entier. Nous avons résolu ce problème en utilisant des valeurs légèrement inférieures à ces extrêmes.

3.2 Approfondissement Itératif (Iterative Deepening)

Pour exploiter au mieux le temps imparti, la recherche est structurée autour d'un approfondissement itératif [3]. L'algorithme explore l'arbre progressivement : à profondeur 1, puis 2, puis 3, et ainsi de suite jusqu'à la limite `depthMax`. À chaque nouvelle itération de profondeur, le temps écoulé est vérifié via notre limite de calcul logicielle (*Soft Bound*). Si cette limite est atteinte en cours d'itération, l'algorithme abandonne l'itération incomplète

en cours et renvoie immédiatement le meilleur coup validé lors de la *dernière itération complétée*. Ce mécanisme garantit qu'un coup fiable et profondément évalué est toujours disponible sans risque de dépassement.

Une amélioration envisagée mais non intégrée dans les délais du projet est l'*Aspiration Windows* : au lieu de relancer chaque itération avec une fenêtre pleine ($\alpha = -\infty$, $\beta = +\infty$), on initialiserait α et β autour du score de la profondeur précédente (par exemple ± 50). En cas d'échec (*fail-low* ou *fail-high*), la fenêtre serait élargie et la recherche relancée. Cette technique peut réduire significativement le nombre de noeuds à explorer lorsque le score est stable entre profondeurs successives.

3.3 Ordonnancement des Coups et Élagage Sélectif

L'efficacité de l'élagage Alpha-Beta dépend directement de la qualité de l'ordonnement des coups [6] : dans le cas idéal, chaque noeud MIN ne considère qu'un seul coup (l'élagage est immédiat). Les deux techniques suivantes, encapsulées dans la classe `MoveOrderer`, ont été implémentées.

Killer Heuristic [7]. À chaque profondeur de l'arbre, on maintient un tableau de 2 *killer moves*, ce sont des coups ayant provoqué une coupure Beta dans un noeud frère à la même profondeur. Ces coups sont essayés en priorité (score fixe `KILLER_SCORE = 900 000`), car il est probable qu'ils causent à nouveau une coupure dans le noeud courant. L'enregistrement suit une file FIFO à 2 slots : le nouveau killer déplace le premier, qui passe en second slot, préservant ainsi un *backup* fiable.

History Heuristic. Un tableau plat `historyTable[36 × 36]` indexé par l'encodage du coup (case de départ × 36 + case d'arrivée, soit 1 296 entrées sur un plateau 6×6) accumule un score à chaque coupure Beta, pondéré par profondeur². Ce poids favorise les coupures profondes, statistiquement plus significatives. Les coups historiquement les plus coupants remontent naturellement en tête de liste sans aucun coût d'allocation.

Tri in-place par insertion. L'ordonnement combine les deux scores (killer prioritaire, puis historique) et trie la liste de coups par insertion directement dans l'objet `ArrayList` renvoyé par `possibleMoves()`. Le tri par insertion est optimal pour de petites listes, le nombre de coups légaux en Escampe ne dépasse généralement pas 30.

Techniques non implémentées. Le *Transposition Table Move* (essai en tête de liste du meilleur coup issu de la table de transposition [7]), le *Null-move Pruning* [8] et le *Late Move Reduction* [9] ont été identifiés comme des améliorations à fort potentiel, mais n'ont pas été intégrés dans les délais du projet. Ils constituent des priorités pour des versions ultérieures.

4 Fonction d'Évaluation (Heuristiques)

L'évaluation des feuilles de l'arbre de recherche se doit d'être précise tout en demeurant très rapide à calculer. Notre heuristique est une combinaison linéaire de plusieurs signaux topologiques.

4.1 Heuristiques Implémentées et Testées

La fonction d'évaluation s'articule autour des métriques suivantes :

1. **Distances d'Attaque et de Défense** : Calcul de la distance de Manhattan entre les Paladins et les Licornes adverses. Plus les attaquants sont proches, meilleur est le score. L'heuristique pénalise la proximité des Paladins adverses à notre Licorne. *Note : une implémentation avancée utilisant une distance en nombre de sauts (BFS ply) intégrant la contrainte des bandes a également été modélisée mais peut être néfaste dû au temps de calcul supplémentaire.*
2. **Échappatoire de la Licorne (Escapability)** : Compte direct du nombre de cases légales de fuite pour la Licorne de chaque joueur. Un adversaire avec peu d'échappatoires est considéré comme dominé positionnellement.
3. **Contrôle des Bandes (Band Control)** : La contrainte du jeu force l'adversaire à partir d'une bande spécifique. Le positionnement de nos Paladins sur des cases de bande 1 (où l'adversaire n'a aucun choix) est fortement valorisé. Cette partie de l'heuristique a été mise en question par les tests du tournoi et des études d'ablation. Elle a été retirée car elle dégradait la performance, probablement à cause d'un trop grand coût ajouté.
4. **Pression Temporelle et Mobilité** : Récompense accordée pour un nombre de coups légaux plus élevé, et forte pénalité lorsqu'un joueur est forcé de passer son tour en raison du mécanisme de contrainte.

Calibration des poids par SPSA. Les neuf poids de la formule ont été calibrés empiriquement via l'algorithme SPSA (*Simultaneous Perturbation Stochastic Approximation*), une méthode de descente de gradient stochastique particulièrement adaptée à l'optimisation de fonctions d'évaluation de jeux [11]. À chaque itération, les paramètres sont perturbés dans deux directions aléatoires ($\pm\delta$), deux parties sont jouées (une dans chaque configuration), et le gradient estimé est utilisé pour mettre à jour les poids. Le processus a été exécuté sur 50 itérations avec 16 paires de parties par itération (32 parties/itération), soit 1 600 parties par run, en parallèle sur tous les coeurs disponibles.

Une étude d'ablation a révélé que le terme de contrôle des bandes ($w_7\mathcal{BC}$) ne contribuait pas positivement : la configuration à 7 paramètres (termes de bande forcés à zéro) surpassait la configuration à 9 paramètres dans les tests croisés. La configuration finale retenue (`createAblated("bandcoverage")`) exclut donc ce terme. Ce résultat est contre-intuitif au vu de l'importance du mécanisme de bandes dans les règles du jeu, mais confirmé statistiquement.

5 Exploration d'une Évaluation par Réseau de Neurones : BandDPER

Bien que l'heuristique manuelle soit performante, les contraintes spatiales fortement non linéaires d'Escampe se prêtent bien à l'apprentissage automatique et il a été montré sur le jeu d'échec [12] ou de Go [14] que les réseaux de neurones surpassent les méthodes traditionnelles. Le modèle **BandDPER** (*Band-Aware Dual-Perspective Equivariant ResNet*) a été conçu comme fonction d'évaluation alternative ou support à l'ordonnancement des coups.

Choix du type de modèle

Modèle	Notes	Recommandation
MLP	Rapide, mais aucune notion spatiale ni raisonnement relationnel	À éviter
CNN	Exploite les motifs spatiaux locaux et l'équivariance par translation	Acceptable, légèrement surdimensionné
NNUE	Mises à jour incrémentales rapides, conception dual-perspective, activations ClippedReLU	Trop conçu pour la taille du problème, mais le dual-perspective et le ClippedReLU sont empruntés
ResNet	Bon si une évaluation de base existe, sans conscience spatiale intrinsèque	Retenu
GNN	Invariant aux symétries, mais graphe dynamique, complexe et lent	À éviter

TABLE 1 : Comparaison des architectures candidates pour la fonction d'évaluation.

La combinaison retenue est un **ResNet dual-perspective avec encodeur CNN spatial**, inspirée du design NNUE [12] (dual-perspective, ClippedReLU, raccourci de sortie direct) et des réseaux siamois [13].

Architecture

BandDPER repose sur six composants principaux : (i) un encodeur CNN spatial siamois à poids partagés, (ii) des canaux d'entrée encodant la topologie des bandes et les contraintes légales, (iii) une carte d'attaquants relative à la licorne (inspirée de l'encodage HalfKP de NNUE), (iv) un tronc résiduel avec ClippedReLU, (v) une injection scalaire des comptes d'échappatoire, et (vi) un raccourci de sortie direct pour le signal de passe forcé.

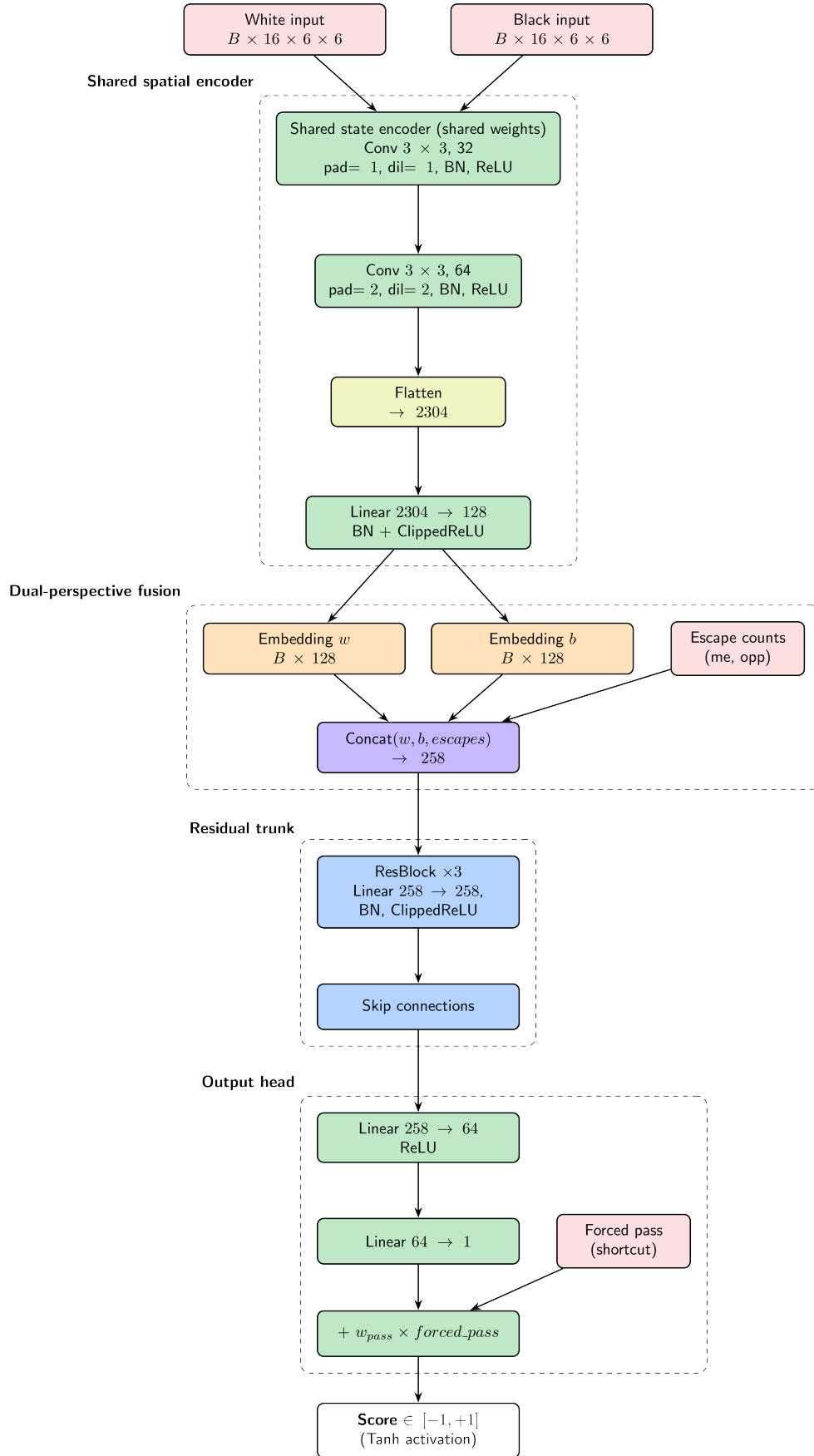


FIGURE 1 : Architecture du réseau BandDPER (encodeur CNN dual-perspective + tronc résiduel à skip connections et bypass direct).

Encodeur spatial partagé (SharedSpatialEncoder). Chaque perspective est encodée séparément via le même réseau (poids partagés, principe des réseaux siamois [13]), sous la forme d'un tenseur $[B, 16, 6, 6]$ représentant 16 canaux de caractéristiques :

Ca- nal	Contenu	Type
0	Positions de mes paladins	Dynamique 0/1
1	Position de ma licorne	Dynamique 0/1
2	Positions des paladins adverses	Dynamique 0/1
3	Position de la licorne adverse	Dynamique 0/1
4	Masque des cases à bande 1	Fixe 0/1
5	Masque des cases à bande 2	Fixe 0/1
6	Masque des cases à bande 3	Fixe 0/1
7	Masque de contrainte de départ	Dynamique 0/1
8–10	Cases d'atterrissage légales (bandes 1, 2, 3)	Dynamique 0/1
11–14	Taux d'occupation par ran- gée/colonne (moi / adv.)	Dynamique [0, 1]
15	Carte d'attaquants relat. à la li- corne	Dynamique 0/1

TABLE 2 : Canaux d'entrée pour chaque tenseur de perspective $[B, 16, 6, 6]$.

Le canal 15 place les paladins adverses dans un référentiel centré sur notre licorne (ancrage en (2,2)), inspiré de l'encodage HalfKP de NNUE [12]. Les canaux 8–10 rendent le mécanisme de forçage de tempo explicitement visible. Les canaux 4–6 (masques de bandes) sont fixes et encodent la topologie non-uniforme du plateau.

L'encodeur projette chaque tenseur en un vecteur de dimension 128 via : $\text{Conv2d}(16 \rightarrow 32, 3 \times 3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv2d}(32 \rightarrow 64, 3 \times 3, \text{dil.} = 2) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Linear}(2304 \rightarrow 128) \rightarrow \text{BN} \rightarrow \text{ClippedReLU}[0, 1]$. La dilatation 2 élargit le champ réceptif à 5×5 , capturant les motifs de mouvement à moyenne portée.

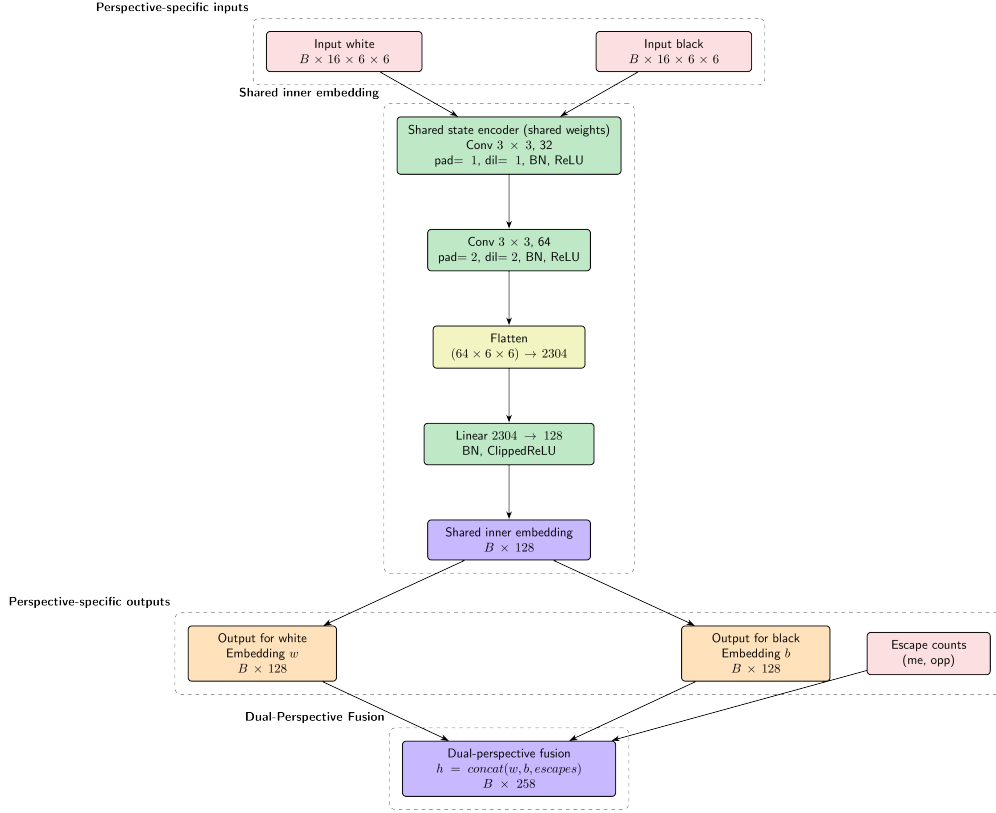


FIGURE 2 : Structure de l'encodeur spatial partagé (Shared Spatial Encoder) utilisé pour chaque perspective (poids partagés).

Fusion et tronc résiduel. Les deux embeddings (256 dimensions) sont concaténés avec deux scalaires normalisés (le nombre de coups légaux de chaque licorne (échappatoire)) formant un vecteur de 258 dimensions. Ce vecteur traverse 3 **ResBlocks** de même dimension (Linear $\rightarrow$ Dropout(0.2) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ BN, puis connexion résiduelle + ClippedReLU). Les 3 niveaux correspondent aux trois abstractions naturelles du jeu : proximité/licorne-relative, interaction de bandes/tempo, forçage/échappatoire.

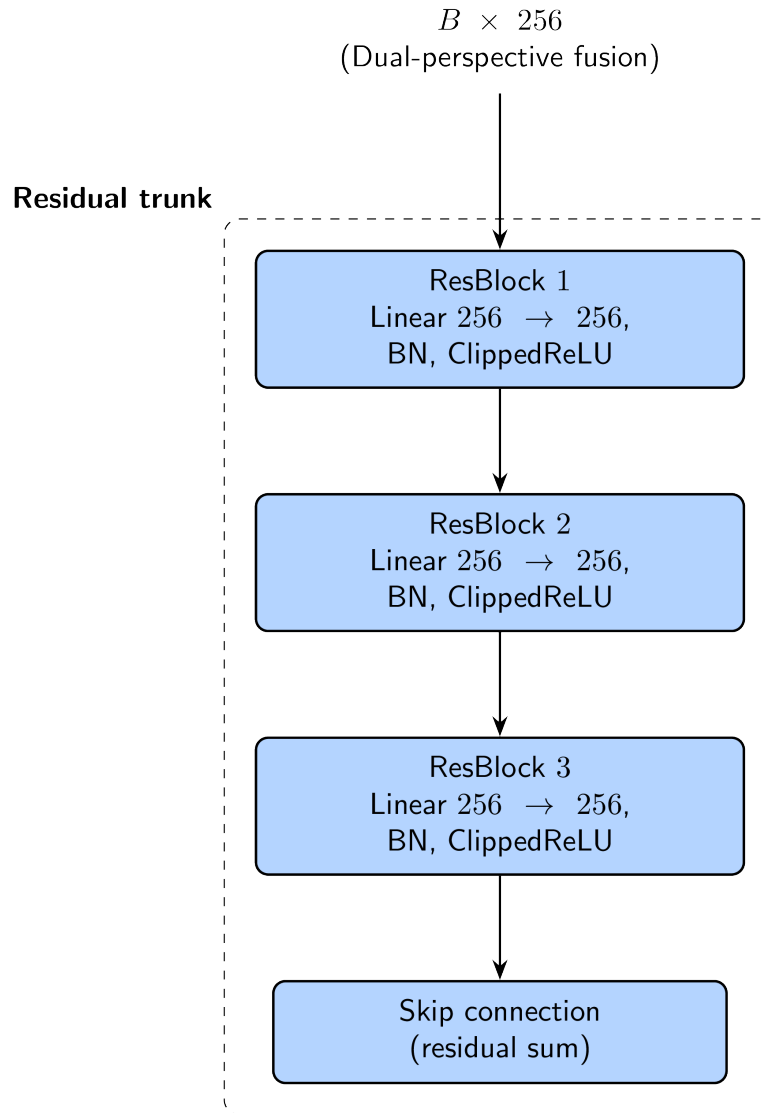


FIGURE 3 : Structure du tronc résiduel avec blocs de correction résiduels (skip connections).

Tête de sortie et raccourci. Linear(258 $\rightarrow$ 64) $\rightarrow$ ReLU $\rightarrow$ Linear(64 $\rightarrow$ 1) $\rightarrow$ tanh, produisant un scalaire dans $[-1, +1]$. Un paramètre appris unique (`w_forced_pass`) s'ajoute directement à la sortie brute sans traverser le tronc, inspiré du raccourci PSQT de Stockfish HalfKAv2 [12]. Ce bypass garantit un gradient fort pour les positions de passe forcé ($\sim 2\text{--}5\%$ du corpus), sans surcharger le tronc.

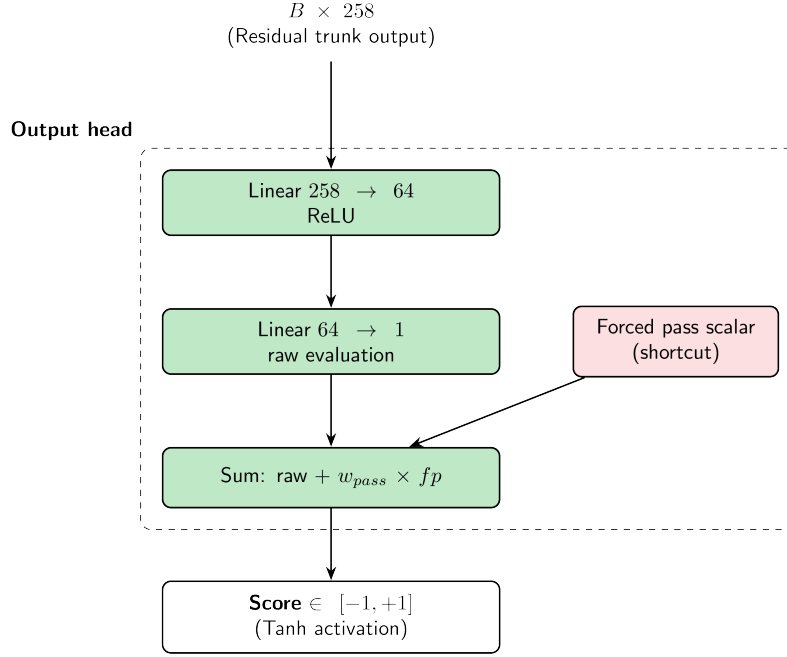


FIGURE 4 : Structure de la tête de sortie avec injection directe du signal de passe forcé (raccourci direct).

Composant	Paramètres
Encodeur partagé (utilisé $\times 2$, poids identiques)	$\sim 312\,000$
Tronc résiduel (3 ResBlocks, dim=258)	$\sim 402\,648$
Tête de sortie	$\sim 16\,577$
Total	$\sim 730\,625$

TABLE 3 : Compte des paramètres par composant ($\approx 2,9$ Mo en float32).

Bilan paramétrique. Une variante allégée (`embed_dim=64`, 2 ResBlocks) atteint $\sim 185K$ paramètres pour une intégration plus rapide.

Génération des données et entraînement

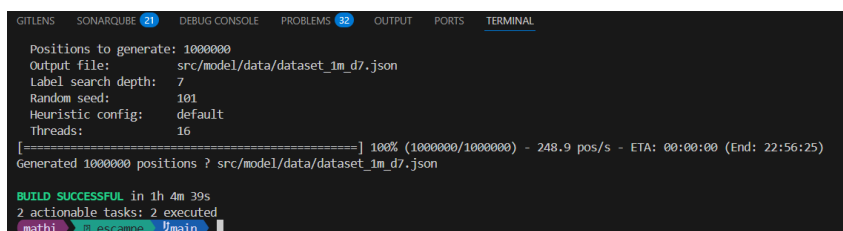
Les étiquettes sont générées par **amorçage Minimax** : pour chaque position échantillonnée en auto-jeu, un alpha-Beta à profondeur 4–6 calcule le score normalisé dans $[-1, +1]$. On vise 50K–100K positions labélisées. Les exemples stockent des triplets (`état`, `bande_dernier_coup`, `label`) pour conserver l'information de contrainte de bande et de passe forcé. L'entraînement utilise une perte MSE, l'optimiseur AdamW ($\text{lr} = 10^{-3}$, weight decay 10^{-4}), un scheduler *CosineAnnealingWarmRestarts* sur 40 époques (batch 256, split 90/10), et un *gradient clipping* à 1.0. Les poids sont initialisés par Xavier uniforme (gain 0,5) pour éviter une saturation précoce des activations ClippedReLU.

Pipeline d'export et optimisations envisagées

Pour l'intégration Java, BatchNorm est replié analytiquement dans les couches adjacentes avant l'export (éliminant les divisions au moment de l'inférence), puis les poids sont sérialisés en JSON ou exportés via ONNX Runtime (latence cible $\sim 1-2$ μ s pour le modèle **Quantisée** INT8 de 25K paramètres quantifiés). Une **tête de politique** ajoutant une distribution sur les ~ 96 coups légaux a été envisagée pour améliorer l'ordonnancement des coups à la racine.

Limites, ressources et statut d'intégration

Nous avons pu générer plusieurs corpus de données :



```

GITLINS  SONARQUBE 21  DEBUG CONSOLE  PROBLEMS 32  OUTPUT  PORTS  TERMINAL
Positions to generate: 1000000
Output file: src/model/data/dataset_1m_d7.json
Label search depth: 7
Random seed: 101
Heuristic config: default
Threads: 16
=====] 100% (1000000/1000000) - 248.9 pos/s - ETA: 00:00:00 (End: 22:56:25)
Generated 1000000 positions ? src/model/data/dataset_1m_d7.json

BUILD SUCCESSFUL in 1h 4m 39s
2 actionable tasks: 2 executed
mathi > escampe /main

```

FIGURE 5 : Export de 1 million de parties à une profondeur de 7

Elle furent ensuite téléchargées sur [Hugging Face](#) pour l'entraînement du modèle BandDPER :

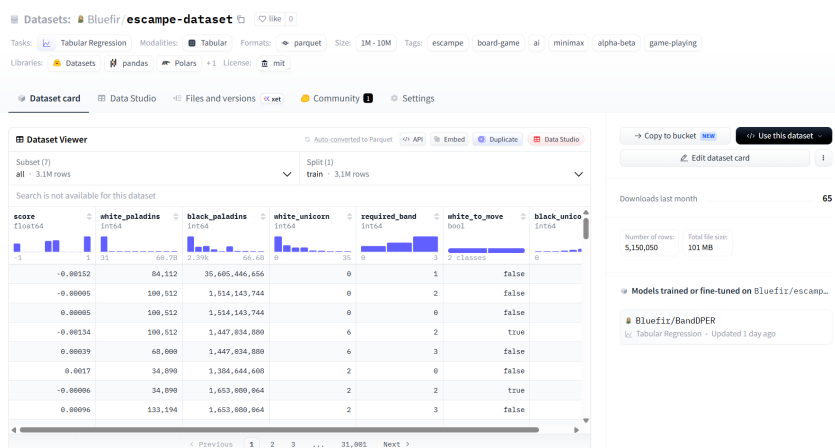


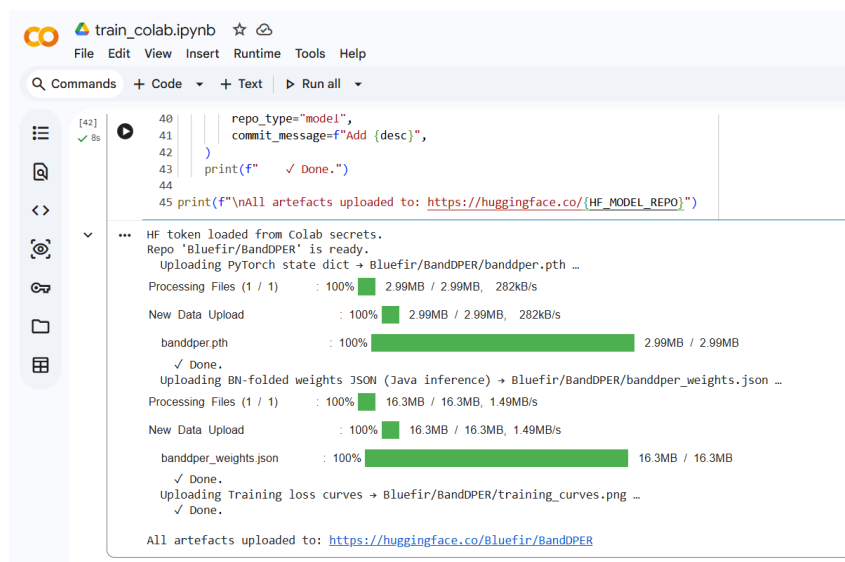
FIGURE 6 : Dataset sur HuggingFace

Les colonnes d'entrée sont les suivantes :

- **white_paladins** : un entier long (bitboard) encodant la position des paladins blancs.
- **white_unicorn** : l'indice de la case occupée par la licorne blanche (de 0 à 35). Logiquement, tends à être plus proche de 0 que de 35.
- **black_paladins** : un entier long (bitboard) encodant la position des paladins noirs.
- **black_unicorn** : l'indice de la case occupée par la licorne noire (de 0 à 35). Logiquement, tends à être plus proche de 35 que de 0.

- `required_band` : le liseré/la bande imposé par le coup précédent (1, 2, 3 ou 0 si aucune contrainte).
- `white_to_move` : un booléen indiquant si c'est au joueur blanc de jouer.
- `score` : le score d'évaluation minimax obtenu à profondeur 5, normalisé entre $-1,0$ et $+1,0$ depuis la perspective du joueur actif.

En faisant un premier entraînement sur 50k positions, on obtient ce modèle (qui fait bien 2.9Mo) :



```

40 repo_type="model",
41 commit_message=f"Add {desc}",
42 )
43 print(f"    ✓ Done.")
44
45 print(f"\nAll artefacts uploaded to: https://huggingface.co/{HF\_MODEL\_REPO}")

```

HF token loaded from Colab secrets.
 Repo 'Bluefir/BandDPER' is ready.
 Uploading PyTorch state dict → Bluefir/BandDPER/banddper.pth ...
 Processing Files (1 / 1) : 100% 2.99MB / 2.99MB, 282KB/s
 New Data Upload : 100% 2.99MB / 2.99MB, 282KB/s
 banddper.pth : 100% 2.99MB / 2.99MB
 ✓ Done.
 Uploading BN-folded weights JSON (Java inference) → Bluefir/BandDPER/banddper_weights.json ...
 Processing Files (1 / 1) : 100% 16.3MB / 16.3MB, 1.49MB/s
 New Data Upload : 100% 16.3MB / 16.3MB, 1.49MB/s
 banddper_weights.json : 100% 16.3MB / 16.3MB
 ✓ Done.
 Uploading Training loss curves → Bluefir/BandDPER/training_curves.png ...
 ✓ Done.
 All artefacts uploaded to: [https://huggingface.co/Bluefir/BandDPER](\"https://huggingface.co/Bluefir/BandDPER\")

FIGURE 7 : Entraînement initial sur 50k positions

En revanche, on observe que le modèle overfit extrêmement :

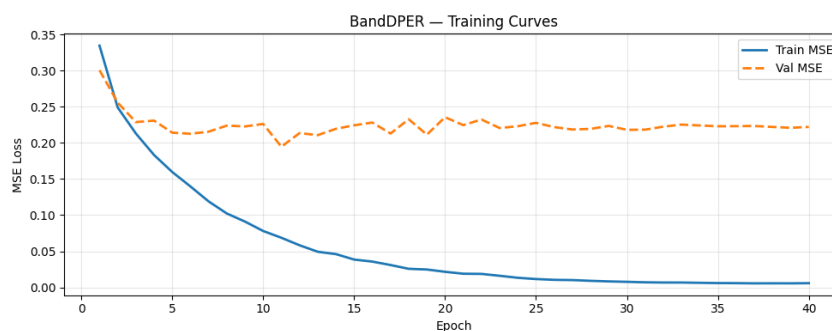


FIGURE 8 : Overfitting du modèle après entraînement initial

Le modèle à tout de même été exporté sur [Modèle Hugging Face](#). Pour régler le problème d'overfitting, j'ai mis à jour le modèle avec un dropout pour régulariser l'entraînement, j'ai augmenté la taille du dataset à 3 million de samples (6 millions en comptant les perspectives inversées). Cependant, je n'ai pas pu observer le résultat faute de temps d'entraînement suffisant :

```
76 print(f"\ntraining complete. Best val loss: {best_val:.5f}")
... Starting training on CUDA...
Epoch 1/40 [Train]: 1% | 30/2725 [08:52<9:37:35, 12.86s/it, loss=0.4727]
```

FIGURE 9 : Entraînement sur 6 millions de positions avec dropout

Pour plus d'informations techniques détaillées sur l'architecture, le processus d'entraînement et l'équivariance du modèle, veuillez consulter le document PDF dédié au modèle BandDPER ([model/BandDPER.pdf](#)).

BandDPER n'a pas été retenu comme heuristique principale : la latence d'inférence depuis Java reste incompatible avec les contraintes de temps par noeud de l'alpha-Beta profond. Son usage le plus prometteur reste l'ordonnancement à la racine (coût ponctuel, non répété), laissé comme piste de développement futur.

6 Stratégie Initiale : Placement et Ouvertures

6.1 Principes de Placement (Opening Book)

L'issue de la partie dépend grandement du positionnement initial. Contrairement aux Échecs, le plateau de départ d'Escampe est défini par les joueurs. Notre agent intègre une bibliothèque de placements de départ (*Opening Book*) garantissant de solides propriétés structurelles :

- **Couverture de toutes les bandes** : Les Paladins doivent couvrir idéalement les bandes 1, 2 et 3. Un déséquilibre offrirait à l'adversaire l'opportunité de cibler une bande où l'agent serait contraint de passer.
- **Dispersion sur les colonnes** : Éviter les grappes de Paladins qui se bloquent mutuellement.
- **Sécurité de la Licorne** : Ne pas murer la Licorne avec ses propres pièces, tout en s'assurant qu'elle ne soit pas au centre géométrique sans couverture, privilégiant les cases "double-bande" pour l'équilibre entre mobilité et défense.

Différentes stratégies (*Balanced*, *Central Pressure*, *Symmetric*) sont tirées aléatoirement pour varier les parties et éviter la prévisibilité. En dehors de la phase de placement, l'agent met à profit les temps de réponse de l'arbitre grâce au **Pondering** [5] : après avoir joué son coup, il prédit le meilleur coup adverse (100 ms de recherche rapide), applique ce coup hypothétique sur une copie du plateau, puis lance en arrière-plan un thread de recherche profonde (budget 4s). Si l'adversaire joue effectivement le coup prédit (*ponder hit*), la réponse est déjà calculée et jouée instantanément, économisant du temps sur le budget principal. En cas d'échec (*ponder miss*), le thread est interrompu et la recherche repart de zéro normalement.

6.2 Stratégie par Phase de Jeu

Une stratégie différenciée par phase peut améliorer significativement les performances :

Début de partie Placement équilibré couvrant plusieurs bandes (1, 2, 3). Licorne protégée derrière des Paladins sans être enfermée. Éviter les couloirs directs vers sa Licorne.

Milieu de partie Jouer le tempo : choisir les cases d'arrivée pour imposer une mauvaise bande adverse ou forcer un passe. Créer des menaces doubles sur la Licorne tout en maintenant une défense active. Limiter les coups qui offrent une bande favorable à l'adversaire.

Fin de partie Chercher les séquences forcantes (capture en 1–3 coups) via une recherche plus profonde grâce à l'approfondissement itératif et à la réduction du budget temporel. En défense, maximiser la mobilité de la Licorne et casser les lignes d'attaque.

7 Gestion du Temps Réel

La communication avec le serveur impose des temps limites stricts sous peine de défaite immédiate. Nous avons implémenté un **TimeManager** [4] hybride avec gestion dynamique des bornes.

- **Estimation du temps par coup** : Le temps de base alloué est calculé en divisant le temps global restant par une estimation dynamique du nombre de coups restants. Cette estimation s'adapte selon la réserve : 40 coups supposés s'il reste beaucoup de temps ($>200s$), puis descend par paliers (25, 15) jusqu'à 8 coups en fin de partie critique.
- **Facteur de Complexité** : Un multiplicateur ajuste le budget temporel en fonction du nombre de coups légaux. Si le joueur n'a qu'un ou deux coups quasi-forcés (contrainte de bande), le budget est drastiquement réduit (facteur de 0.3) pour économiser du temps. Dans une position très ouverte (≥ 20 coups), le budget augmente (facteur 1.5).
- **Soft Bound / Hard Bound** :
 - La *Soft Bound* (limite logicielle) correspond au budget temporel de base ajusté. Une fois franchie, la boucle d'*Iterative Deepening* s'interrompt pour renvoyer le meilleur coup de la dernière profondeur complète.
 - La *Hard Bound* (limite matérielle) agit comme une sécurité absolue, fixée à $3\times$ la *Soft Bound* (avec un plafond strict à un cinquième du temps total restant). Si elle est atteinte en plein milieu de l'évaluation d'un noeud (lors des appels récursifs du Minimax), l'algorithme coupe instantanément la branche et retourne l'évaluation heuristique courante, empêchant ainsi le dépassement de temps fatal.

8 Analyse des Performances et Tests

Les performances de l'IA ont été évaluées via un moteur de test autonome opposant plusieurs versions de notre agent.

8.1 Évaluation Elo et Test SPRT

L'amélioration des heuristiques et l'optimisation des structures de données ont été quantifiées via un système de tournoi automatique (**TournamentRunner**) opposant de multiples configurations de notre agent. Le classement repose sur le rating Elo avec mise à jour $R' = R + K(S - E)$, où $E = 1/(1 + 10^{(R_B - R_A)/400})$ et $K = 32$. Les configurations testées incluent différentes profondeurs (2 à 6), les deux algorithmes de recherche

(AlphaBetaKH et NegamaxKH), les trois variants d'heuristique (Default, SimplexFull, BandCoverage-ablated) et l'activation ou non du Pondering.

Pour valider statistiquement qu'une modification apporte un gain Elo réel avant de la valider, nous utilisons le **SPRT** [11] (*Sequential Probability Ratio Test*). Le SPRT compare deux hypothèses : H_0 (les deux agents sont équivalents, $Elo = 0$) contre H_1 (l'un est meilleur, $Elo > 0$). Le test s'arrête dès que les données accumulées sont suffisantes pour accepter ou rejeter H_0 au seuil de confiance fixé (95%), évitant de jouer des parties inutiles. Ce processus itératif, inspiré des pratiques de Stockfish, garantit que seules les améliorations réelles progressent vers la version finale.

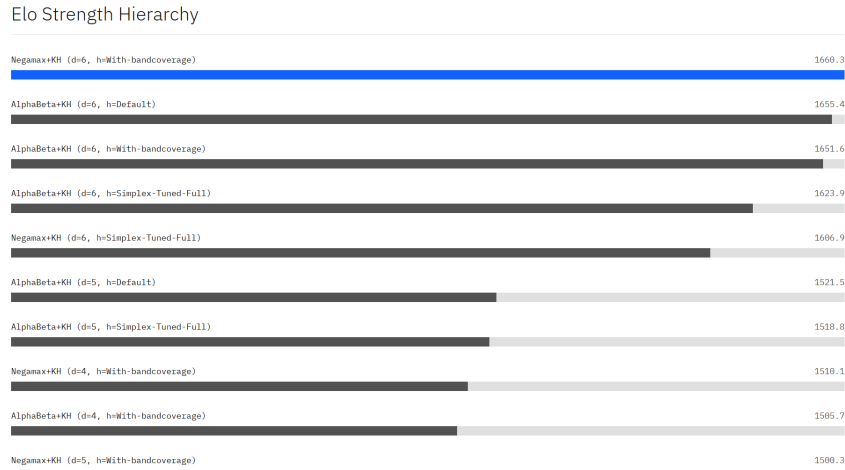


FIGURE 10 : Évolution du classement Elo des différentes configurations testées (tests SPRT) sur 2000 parties par matchup.

Le test n'est significatif que lorsque le nombre de parties jouées dépasse un certain seuil (généralement plusieurs centaines ou milliers) pour réduire l'impact de la variance. A noter aussi que certaines améliorations sont invisibles à faible profondeur (ex : ordonnancement des coups) et ne montrent leur plein potentiel qu'à partir de la profondeur 4 ou 5, d'où l'importance de tester sur plusieurs profondeurs. Voici le premier test réalisé :

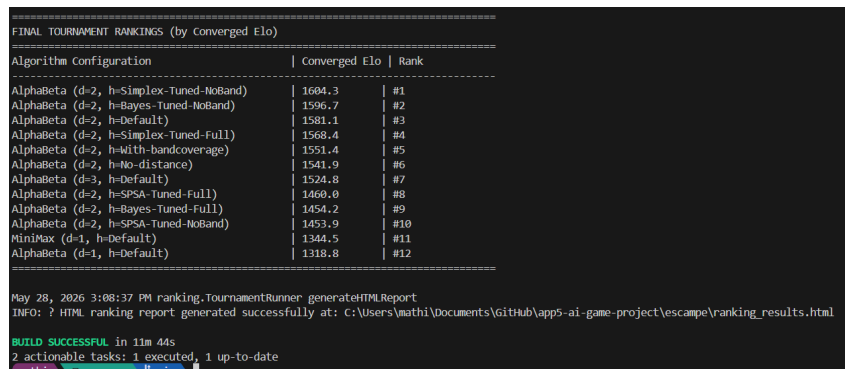


FIGURE 11 : Premier test de classement Elo après implémentation des fonctionnalités principales et heuristiques calibrées.

8.2 Validation de la Conduite Autonome

Les protocoles de communication avec l'arbitre ont été validés localement en s'assurant qu'aucun dépassement de délai n'intervenait et que tous les coups générés restaient

strictement conformes aux règles topologiques du jeu.

9 Bilan des Fonctionnalités et Statut du TODO

Afin de mesurer le travail accompli au cours du projet et de structurer les développements futurs, un suivi rigoureux a été maintenu sous la forme d'un tableau d'objectifs (TODO). Cette section dresse le bilan des fonctionnalités implémentées, validées et de celles envisagées comme perspectives d'amélioration.

9.1 Fonctionnalités Entièrement Implémentées et Validées

- **Moteur de Recherche et Pruning** : Implémentation complète de l'algorithme **Negamax** avec élagage Alpha-Beta symétrique. L'approfondissement itératif (*Iterative Deepening*) a été déployé avec succès, garantissant une flexibilité absolue face aux limites de temps de réflexion.
- **Gestion Fine du Temps** : Mise en oeuvre du **TimeManager** avec un double niveau de sécurité (*Soft Bound* pour clore proprement la recherche à la fin d'une profondeur donnée, et *Hard Bound* coupant instantanément l'exécution récursive du minimax en cas d'urgence). Le temps est mesuré à haute résolution via **System.nanoTime()**.
- **Ordonnancement des Coups** : Intégration d'un ordonnanceur de coups (**MoveOrderer**) combinant l'heuristique des coups tueurs (*Killer Heuristic* avec FIFO à 2 slots) et l'heuristique de l'historique (*History Heuristic* pondérée par la profondeur au carré). Le tri est réalisé *in-place* pour minimiser l'impact mémoire.
- **Heuristiques Linéaires Calibrées** : Conception d'une formule d'évaluation combinant la proximité d'attaque, la proximité de défense, la mobilité/escapabilité de la licorne, le contrôle de bande et la pression de tempo. Ces poids ont été calibrés par optimisation stochastique SPSA. L'analyse d'ablation a conduit à exclure le terme de contrôle des bandes pour de meilleures performances (variant **BandCoverage-ablated**).
- **Bibliothèque d'Ouvertures et Réflexion en Temps Masqué** : Intégration d'un livre d'ouvertures diversifié (Balanced, Central Pressure, Symmetric, Flanking, etc.) assurant une bonne couverture de départ. Déploiement du mécanisme de *Pondering* via un thread d'arrière-plan analysant le coup adverse supposé pendant son temps de réflexion (avec gestion des cas de *ponder hit* et *ponder miss*).
- **Optimisations Système et Représentation** : Passage d'une représentation matricielle classique à des Bitboards (long de 64 bits en Java), élimination des vérifications **instanceof** et implémentation du mécanisme *Make-Unmake* pour éviter le clonage d'objets et stabiliser le Garbage Collector.
- **Conception du Modèle de Deep Learning** : Modélisation, génération de données (Minimax bootstrapping à profondeur 5, 50K-100K positions), entraînement sous PyTorch du modèle BandDPER, et intégration des mécanismes d'export (repliement analytique de BatchNorm).

9.2 Fonctionnalités Conçues, Rejetées ou Non Retenues

- **Contrôle de Bande dans l'Heuristique** : Bien que central dans les règles du jeu, le contrôle manuel des bandes a été rejeté après tests SPRT et ablation, car il

nuisait statistiquement au classement Elo global de l'agent.

- **Utilisation Directe du Réseau de Neurones en Jeu** : Le réseau BandDPER n'a pas été retenu comme fonction d'évaluation principale dans l'arbre en raison de sa latence d'évaluation trop élevée par rapport à l'heuristique linéaire optimisée en Java. Il est en revanche préconisé pour l'ordonnancement à la racine.

9.3 Perspectives et Travaux Futurs (TODO Ouverts)

- **Table de Transposition et Zobrist Hashing** : Le schéma d'encodage par hachage Zobrist a été conçu mais n'a pas été finalisé dans l'exécution principale. Son intégration permettrait de stocker les scores d'évaluation des noeuds déjà parcourus et d'identifier instantanément les répétitions de positions (détection de boucles de jeu).
- **Élagages Sélectifs Avancés** : Intégration de techniques d'élagage plus agressives comme le *Null-Move Pruning* (pour éliminer rapidement les branches très favorables), le *Late Move Reduction* (LMR) pour réduire la profondeur d'exploration des coups moins prometteurs, et la recherche de quiescence (*Quiescence Search*) aux feuilles.
- **Aspiration Windows** : Utilisation de fenêtres de recherche resserrées au lieu d'une fenêtre de recherche infinie lors de l'approfondissement itératif, pour maximiser l'élagage lorsque l'évaluation est stable.
- **Inférence ONNX Quantifiée** : Intégration finale de l'exécuteur ONNX Runtime pour charger le modèle BandDPER quantifié en INT8 et l'évaluer en temps réel sous une latence ciblée de 1 à 2 μ s par coup.

10 Conclusion et Difficultés Rencontrées

La mise au point de cette IA pour le jeu d'Escampe a été un défi algorithmique enrichissant. Les principales difficultés ont résidé dans :

- La compréhension initiale des symétries brisées du plateau (l'absence d'uniformité due aux bandes asymétriques complexifie fortement la pré-détermination de patterns géométriques réguliers).
- Le débogage subtil inhérent à l'algorithme Alpha-Beta, notamment lors de l'intégration de l'ordonnancement dynamique des coups et de la gestion asynchrone des limites temporelles (*Hard bounds*).
- La conception d'une heuristique qui ne se laisse pas piéger par l'effet d'horizon, exacerbé dans Escampe par les réactions en chaîne de la mécanique de passe obligatoire.

L'agent final représente une synthèse robuste entre techniques classiques de la programmation de jeux à information parfaite et optimisation mathématique de topologie.

Références

- [1] Chess Programming Wiki, *Alpha-Beta*, <https://www.chessprogramming.org/Alpha-Beta>
- [2] Chess Programming Wiki, *Bitboards*, <https://www.chessprogramming.org/Bitboards>
- [3] Chess Programming Wiki, *Iterative Deepening*, https://www.chessprogramming.org/Iterative_Deepening
- [4] Chess Programming Wiki, *Time Management*, https://www.chessprogramming.org/Time_Management
- [5] Chess Programming Wiki, *Pondering*, <https://www.chessprogramming.org/Pondering>
- [6] Chess Programming Wiki, *Move Ordering*, https://www.chessprogramming.org/Move_Ordering
- [7] Chess Programming Wiki, *Killer Heuristic*, https://www.chessprogramming.org/Killer_Heuristic
- [8] Chess Programming Wiki, *Null Move Pruning*, https://www.chessprogramming.org/Null_Move_Pruning
- [9] Chess Programming Wiki, *Late Move Reductions*, https://www.chessprogramming.org/Late_Move_Reductions
- [10] Chess Programming Wiki, *Zobrist Hashing*, https://www.chessprogramming.org/Zobrist_Hashing
- [11] Chess Programming Wiki, *Match Statistics (SPRT)*, https://www.chessprogramming.org/Match_Statistics
- [12] Yu Nasu, *Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi*, The 28th World Computer Shogi Championship Appeal Document, 2018. <https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html>
- [13] J. Bromley et al., *Signature Verification using a Siamese Time Delay Neural Network*, NeurIPS, 1993. <https://papers.nips.cc/paper/1993/hash/0d4262ad58b4d3866d5aa0f4f9e1c06b-Abstract.html>
- [14] D. Silver et al., *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, arXiv:1712.01815, 2017. <https://arxiv.org/abs/1712.01815>